



Compiladores

Aula 5: Construção do analisador léxico

Prof. Alex Torquato Souza Carneiro
alex.carneiro@impacta.edu.br

Agenda

- Elementos da implementação de um scanner
- Maximal munch
- Leitura em blocos
- Casos de erro

Elementos da implementação de um scanner

- Como transformar uma especificação de token em um scanner confiável?
- O scanner deve ser uma ponte entre a regra escrita e o reconhecimento operacional de lexemas.

01 Do padrão ao scanner

Como expressões regulares e autômatos viram regras executáveis.

02 Decisões léxicas

Maior casamento, prioridades, lookahead e ações semânticas.

03 Entrega da Etapa 1

Arquitetura, testes, erros e contrato de integração com o parser.

Elementos da implementação de um scanner

- As decisões léxicas podem retornar um token, converter um valor, atualizar a tabela de símbolos, ignorar o lexema ou emitir um diagnóstico.



Nosso analisador léxico deverá integrar reconhecimento, ações e controle da entrada.

Elementos da implementação de um scanner

- Cada elemento responde a uma pergunta diferente — e o contrato entre eles permanece estável.

Leitura



Como fornecer caracteres e posição?

Reconhecimento



Qual padrão casa com a entrada?

Classificação



Qual token representa o lexema?

Atributos



Qual valor e localização transportar?

Erros



Como diagnosticar e recuperar?

Elementos da implementação de um scanner

- A tabela de tokens é o contrato do **scanner** e será consumida pelo **parser**.

Classe	Padrão sugerido	Ação
IDENT	[A-Za-z_][A-Za-z0-9_]*	atributo textual
INT_LIT	[1-9][0-9]*	converter para inteiro
FLOAT_LIT	([1-9][0-9]*)?\.[0-9]+	converter para real
OPERADORES	+ - * / % == != <= >=	token correspondente
IGNORADOS	espaços; //...; /*...*/	ignorar e atualizar linha

Exemplos de itens para a tabela de tokens.

Maximal munch

- Quando regras compartilham prefixos, a estratégia precisa ser determinística.
- MAXIMAL MUNCH:

Escolha o lexema mais longo possível.



Maximal munch

- No caso de empate entre regras, definimos uma prioridade explícita:
 1. operadores **compostos** antes dos simples
 2. palavra reservada antes de **identificador**
 3. ordem documentada no scanner

Maximal munch

- O scanner precisa escolher uma regra, produzir atributos e manter a posição exata na entrada.

```
int total = valor + 10;
```

```
INT IDENT ASSIGN IDENT PLUS INT_LIT
```

Reconhecer

Encontrar o maior prefixo da entrada que pertence a uma classe léxica.

Classificar

Distinguir palavra reservada, identificador, literal, operador ou delimitador.

Agir

Emitir token + atributo + linha/coluna; ignorar ou diagnosticar quando necessário.

Leitura em blocos

- A leitura em blocos reduz o custo de chamadas ao sistema e sustenta o maior casamento.

BUFFER A

`int total = valor`



BUFFER B

`+ 10; EOF`

lexemeBegin: início do lexema atual

forward: avança para testar o casamento

sentinela: detecta fim com menos testes

Casos de erro

- O scanner deve localizar o problema, preservar contexto e avançar de modo controlado.

x@2

símbolo desconhecido

posição do @

char c = 'a;

literal não terminado

fim da entrada

12.

número malformado

regra de real

/* sem fechamento

comentário incompleto

EOF